Ammeter Testing Exercise - Documentation

Environment

- Python 3.9.6 (macOS system `python3`), run inside a virtual environment at `.venv/`.
- Setup:

```
 python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

- Run the emulators and take one measurement from each: `.venv/bin/python main.py`
- Run the tests: `.venv/bin/python -m unittest discover -s tests -t .` (standard-library `unittest`, no extra dependency)

Libraries installed

Installed from `requirements.txt`:

Package	Version
numpy	2.0.2
scipy	1.13.1
matplotlib	3.9.4
seaborn	0.13.2
PyYAML	6.0.3
pandas	2.3.3

`pytz` was also installed automatically as a dependency of `pandas`.

Bugs found and fixed

1. `main.py` returned no data

- **Symptom:** running `main.py` started the servers and exited without printing any measurement.

- **Cause:** the three `request_current_from_ammeter(...)` calls were commented out, and the commands in them were truncated (`b'MEASURE_GREENLEE'` , `b'MEASURE_ENTES'` , `b'MEASURE_CIRCUTOR'`). The server only replies when the received bytes equal its full command exactly. On a mismatch it sends nothing and closes the connection, so the client prints "No data received."
- **Fix:** uncommented the calls and used the full commands:

Ammeter	Port	Command
Greenlee	5001	`MEASURE_GREENLEE -get_measurement`
ENTES	5002	`MEASURE_ENTES -get_data`
CIRCUTOR	5003	`MEASURE_CIRCUTOR -get_measurement`

- **File:** `main.py` . It has since been rewritten to read the ports from `config/config.yaml` , start the emulators with `start_in_background` , and print `framework.measure_all()` . The fixed `sleep(5)` was replaced by waiting until each port accepts connections.

2. "Address already in use" when restarting

- **Symptom:** running `main.py` a second time shortly after the first raised `OSError: [Errno 48] Address already in use` in all three server threads, followed by `ConnectionRefusedError` in the client.
- **Cause:** the connections from the previous run left the ports in TCP `TIME_WAIT` , and the server sockets did not set `SO_REUSEADDR` , so `bind()` was refused.
- **Fix:** set `SO_REUSEADDR` before `bind()` in `AmmeterEmulatorBase.start_server` . The option is applied only when `sys.platform != "win32"` : on Windows it allows several processes to bind the same live port, and Windows does not block binding to `TIME_WAIT` ports, so it is not needed there.
- **File:** `Ammeters/base_ammeter.py` .
- **Verified:** three consecutive runs each returned a reading from all three ammeters (on macOS; Windows was not tested).

3. CIRCUTOR command differed from the README

- **Symptom:** the emulator required `MEASURE_CIRCUTOR -get_measurement -current` , while the README and the commented config give `MEASURE_CIRCUTOR -get_measurement` .
- **Cause:** an undocumented `-current` suffix in the emulator.
- **Fix:** removed the suffix from the emulator so the command matches the README, the config, and Greenlee's `-get_measurement` pattern. The client call in `main.py` was updated to match.
- **Files:** `Ammeters/Circutor_Ammeter.py` , `main.py` .

4. `test_framework.py` crashed on import

- **Symptom:** importing `src/testing/test_framework.py` raised `NameError: name 'Dict' is not defined` .

- **Cause:** `Dict` was used in an annotation without being imported from `typing`; `run_test` was also an empty stub.
- **Fix:** rewrote the module on top of the new measurement API (see Design decisions). `run_test` raises `NotImplementedError` until sampling is implemented.
- **File:** `src/testing/test_framework.py`.

Known inconsistencies (not yet addressed)

- The README lists ports 5000, 5001 and 5002; the code uses 5001, 5002 and 5003. The code is internally consistent.
- ENTES uses `-get_data` while the others use `-get_measurement`. This matches the README, so it is treated as intentional; the framework should read each command from configuration instead of assuming a shared one.
- The README mentions `AmmeterTester.py`, `Ammeters/main.py` and `examples/run_test.py`; the actual files are `src/testing/test_framework.py`, `main.py` and `examples/run_tests.py`.
- `src/utils/logger.py` never attaches a handler, so nothing is logged.
- The `testing.sampling`, `analysis` and `result_management` sections of `config/config.yaml` are still empty (only `ammeters` is filled).
- `Ammeters/client.py` is no longer used by `main.py`; it is left as supplied.

Design decisions

1. Unified Measurement API (`src/testing/ammeter_api.py`)

- **One result format.** `MeasurementResult` (ammeter, success, value, unit, UTC timestamp, duration, error) is returned for every ammeter, for successes and failures alike. `to_dict()` makes it JSON-ready for the later archiving step.
- **`measure()` never raises.** Connection errors, timeouts, empty replies, non-numeric replies and NaN/infinity are all reported through the result (`success=False` plus an error message), so a sampling loop can record a failure and keep going.
- **Template method.** `AmmeterInterface.measure()` handles timing, parsing and error reporting; a subclass only implements `_read_raw()`. A new ammeter type or a transport other than TCP needs one small class.
- **Configuration-driven.** Ports and commands come from `config/config.yaml`, so differences between ammeters (for example ENTES's `-get_data`) live in config rather than code. `host` and `timeout_seconds` are optional per ammeter. Invalid config raises `AmmeterConfigError` with a clear message.
- **Robust socket read.** The client reads until the server closes the connection (capped at 4096 bytes) instead of assuming one `recv` returns the whole reply.

- **Framework entry point.** `AmmeterTestFramework` provides `measure(type)`, `measure_all()` and `get_ammeter(type)`. Type names are case-insensitive and an unknown type lists the valid ones.
- **Shared helpers** in `src/utils/Utils.py`: `find_free_port`, `wait_for_port` and `start_in_background`, used by `main.py` and the tests.

Tests (`tests/test_ammeter_api.py`)

- Failure modes (connection refused, timeout, empty, non-numeric and non-finite replies) are tested against a small fake server, because the real emulators cannot produce bad replies on demand.
- Config validation is tested directly.
- Integration tests run the framework against the three real emulators on free ports, using the commands from `config.yaml`.

Not implemented yet

Measurement sampling, result analysis, result archiving, accuracy assessment and the bonus challenges.